FLIPSIDE · COMPONENT SPEC · M1

`flip seal` v1

Before the pull request is reviewed, the author's machine fixes every message of the agent session into one 32-byte fingerprint and writes it into git. Later, any single quote from that session can be shown to the reviewer with a short proof that it was there all along, while the rest of the session stays on the author's machine.

Awaiting approval | Math prototyped: proofs verify for every tree size 1 to 69 | Python stdlib only | Claude Code sessions

What a verified quote proves

- These exact bytes were in the session when it was sealed.
- They belong to step *s*, chunk *c*, with the tools and files the manifest lists.
- The seal was in the commit the question named, so editing the session afterwards cannot change the answer.
- How many other sealed steps touched the same file and stayed hidden.

What it does not prove

- That the session file is genuine. An author who forges a session before sealing is out of scope.
- That the quote fully answers the question. The coverage line shows what was withheld.
- That this session wrote the code. Work done outside the agent has no session evidence.

PIPELINE

From session file to seal

1

Read

Load the Claude Code session JSONL. Keep main-thread `user` and `assistant` lines only.

2

Steps

One step per message. Assistant lines that share `message.id` merge; each tool result is its own step.

3

Chunks

NFC, CRLF to LF, then at most 300 characters per chunk, cut at the last space or newline.

4

Leaves

Each chunk plus its metadata, as canonical JSON, salted and hashed with prefix `0x00`.

5

Tree

RFC 9162 Merkle tree over all leaves, inner nodes hashed with prefix `0x01`.

6

Commit

An empty seal commit carries the root, leaf count and manifest hash as a `Flipside-Seal:` trailer.

7

Room

When `#pr-142` opens, author-agent posts the seal line and manifest. Switch timestamps it.

☐ stays on the author's machine ☐ public: git and the Switch room

RULES

Exact construction

Steps and files

A step's text is its text blocks verbatim, then each tool call as `[tool Edit]` plus the canonical JSON of its input. A tool result's text is its text content; an image becomes `[image sha256:…]`.

`files` comes only from structured tool input: `file_path`, `notebook_path`, `path`. Paths are repo-relative POSIX; anything outside the repo becomes `<external>`. A tool result inherits the files of its call through `tool_use_id`. Bash commands are sealed as text but tag no files.

Thinking blocks are not sealed.

Hashes

```
# K: 32 random bytes, .git/flipside/<root>.key,
mode 0600, never sent
leaf  = canonical_json({v, session, step, chunk,
                        role, tools, files, tex
t})
salt  = HMAC-SHA256(K, "flipside/salt/v1" ||
                       canonical_json([session,
step, chunk]))
L(i)  = SHA-256(0x00 || salt || leaf)
N     = SHA-256(0x01 || left || right)
split = largest power of two < n      # RFC 9162
§2.1.1
root  = MTH(L(0) … L(n-1))
manifest = canonical_json([leaf minus text, plus
i, for each leaf])
```

Canonical JSON: sorted keys, no spaces, UTF-8, integers and strings only, so every implementation produces the same bytes.

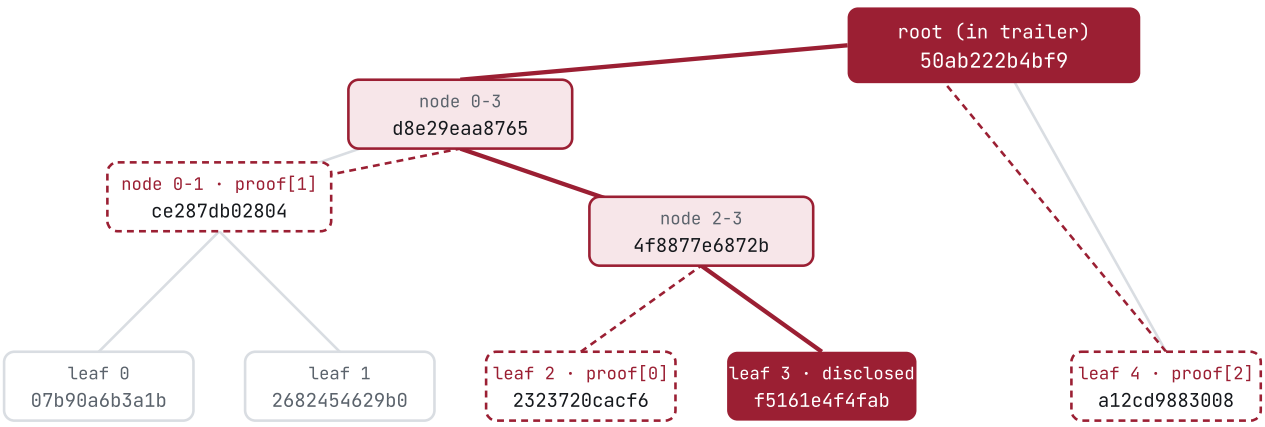
WORKED EXAMPLE · REPRODUCE WITH `PYTHON3 DOCS/SEAL/EXAMPLE.PY`

A five-step session

The data team's agent adds CSV export and quietly raises a session timeout. These are real hashes, computed with a demo key of 32 zero bytes so anyone can reproduce them.

LEAF	STEP	ROLE	TOOLS	FILES	TEXT	L(I)
0	0	user	-	-	Add CSV export to the reports page.	07b90a6b3a1b
1	1	assistant	Edit	`reports/export.py`	Adding export_csv() to reports/export.py.	2682454629b0

LEAF	STEP	ROLE	TOOLS	FILES	TEXT	L(I)
2	2	tool_result	Bash	-	FAILED tests/test_export.py::test_export_csv - SessionExpired	2323720cacf6
3	3	assistant	Edit	`auth/session.py`	test_export_csv failed with SessionExpired; raising SESSION_TTL to 86400 in auth/session.py	f5161e4f4fab
4	4	assistant	-	-	CSV export works and the tests pass.	a12cd9883008



Solid red: hashed by the reviewer from the disclosed chunk.
Dashed: the 3 sibling hashes sent as the proof. Grey: never sent.

Five leaves split 4 + 1, as RFC 9162 requires: the odd leaf is promoted, never duplicated. Because every leaf is salted, the dashed sibling hashes tell the reviewer nothing about the text of leaves 0, 1, 2 or 4.

Written to git

```
flipside: seal session

Flipside-Seal: v1 root=50ab222b4bf9c20399f20e3f3
22f39841fa072c8ed4806e1faa890ff757c6f03 n=5 mani
fest=6311c5cd173fc97e4bf0fa30208a890aa52185ad684
5bb9b08680bb1509e3060
```

`git commit --allow-empty --trailer` (git 2.32 or later). `--amend` adds the trailer to HEAD instead, for a branch that is not pushed yet.

Disclosure sent after approval

```
{"v":1, "commit":"a41b2c7…", "i":3, "n":5,
 "salt":"29c3111716183cdc…3aef2a",
 "leaf":{"chunk":0,"files":["auth/session.py"],
         "role":"assistant","session":0,"step":
3,
         "text":"test_export_csv failed with …",
         "tools":["Edit"],"v":1},
 "proof":["2323720cacf6…","ce287db02804…",
          "a12cd9883008…"]}
```

The `text` field is the full chunk; it is shortened here only for width.

REVIEWER SIDE

`flip verify` checks, in order

```
flip verify --commit a41b2c7 --manifest manifest.json disclosure.json
```

1	`NO_SEAL`	The named commit has no `Flipside-Seal:` trailer, or more than one.
2	`WRONG_COMMIT`	The disclosure names a different commit from the question. A re-seal in a later commit cannot answer an earlier question.
3	`NOT_IN_PR`	The commit is not an ancestor of, or equal to, the PR head (`git merge-base --is-ancestor`).
4	`MANIFEST_HASH`	SHA-256 of the manifest does not match the trailer.
5	`MANIFEST_MISMATCH`	The leaf's metadata differs from manifest entry *i*.
6	`BAD_SIZE`	$i \geq n$, or the proof length is wrong for that index and size.
7	`PROOF_FAIL`	Hashing up from the leaf (RFC 9162 §2.1.3.2) does not reach the root. A forged or edited chunk stops here.
✓	`OK`	`leaf 3 (step 3, chunk 0) against seal 50ab222 in commit a41b2c7`

The reviewer-agent then computes coverage itself from the public manifest, so it does not have to trust the author's count: `1 sealed step touched auth/session.py: 1 disclosed, 0 withheld.` On the author's side, `flip prove` refuses with `SESSION_CHANGED` if the session file's SHA-256 differs from the one recorded at seal time.

THREATS

What each rule defends

ATTACK	STOPPED BY	RESULT
Edit a quote after sealing	Leaf hash and inclusion proof	`PROOF_FAIL`
Edit the session, re-seal, answer again	Questions pin the seal commit	`WRONG_COMMIT`
Guess a short hidden chunk from sibling hashes	256-bit HMAC salt per leaf	Guessing needs the salt
Pass an inner node off as a leaf	`0x00` / `0x01` prefixes	`PROOF_FAIL`
Duplicate-last-leaf tree tricks	RFC 9162 split, *n* in the trailer	`BAD_SIZE` or `PROOF_FAIL`
Show one harmless step, hide three others on the same file	Public manifest, reviewer-side coverage	Withheld steps listed
Backdate the seal	Switch room post timestamp, plus the push	Two independent records
Forge the session before sealing	Not stopped. Out of scope, stated in Limits.	

The manifest is public, so it reveals the step count, roles, tool names, repo-relative paths and chunk counts (length to the nearest 300 characters). It never contains text.

FOR APPROVAL

Decisions

ID	CHOSEN	REJECTED, AND WHY
D1	One step per message; assistant blocks merge by `message.id`	One step per content block: splits the reasoning from the edit it explains and breaks the file check
D2	Thinking blocks not sealed	Sealing them: often redacted or signed, and the most private part of the session
D3	Chunks of at most 300 characters, cut at whitespace	Fixed windows cut words mid-quote; sentence splitting depends on language
D4	Metadata (step, tools, files) inside each leaf	Text-only leaves: the reviewer could not verify that the step touched the file
D5	Salts derived from one local key by HMAC	A stored random salt per leaf means thousands of secrets; no salt makes short chunks guessable
D6	RFC 9162 Merkle tree with SHA-256	Duplicate-last-leaf trees: known mutation bug (CVE-2012-2459)
D7	Empty seal commit with a trailer; `--amend` opt-in	git notes are not pushed by default and GitHub hides them; amending by default rewrites pushed history
D8	Public manifest posted in the room, its hash in the trailer	Committing it adds a file to the PR the owning team merges
D9	Verify against the commit the question names	Verifying against HEAD lets a re-seal win
D10	Python standard library only	A Merkle package adds a dependency to the verification path
D11	Claude Code sessions in M1; Codex adapter next	Both at once: two parsers before one works end to end

GATES

M1 is done when

G1	Inclusion proofs verify for every leaf of every tree size from 1 to 256. Prototype passes 1 to 69.
G2	Flipping any single bit of the text, salt, a sibling hash, the index, *n* or the root makes verify fail with the right code.
G3	The example on this page reproduces root `50ab222b…` and manifest `6311c5cd…` byte for byte. Prototype passes.
G4	Sealing a real Claude Code session twice with the same key gives the same root.

G5	End to end in git: an edited quote gives `PROOF_FAIL`; a re-seal after the question gives `WRONG_COMMIT`; an edited session gives `SESSION_CHANGED`.
G6	ruff and mypy clean; `make test` green on two consecutive runs.

Approving this also changes the pitch

1. The coverage line counts steps, not leaves, and the reviewer-agent computes it from the manifest instead of the author-agent.
2. The trailer goes on a dedicated seal commit, not on whatever the PR head commit is.
3. The seal line is also posted in `#pr-142` when the room opens, which answers the Codex judge's "no independent record" point.
4. The pitch example's seal id becomes `50ab222`, to match this page.

Flipside · `docs/seal/` · prototype math: `docs/seal/example.py`